1. What is VeriLingo?

VeriLingo is a multilingual, general-purpose news/claim verification system.

The key word is **general-purpose**.

It isn't designed only for sports, politics, disasters, finance, etc. A user can enter essentially any factual claim:

"India has won a FIFA World Cup."

or

"Earth has a moon."

or

"Earth has purple oceans."

VeriLingo searches the **live web**, retrieves relevant evidence, analyzes that evidence against the claim, and produces:

SUPPORTED

CONTRADICTED

UNCERTAIN

along with:

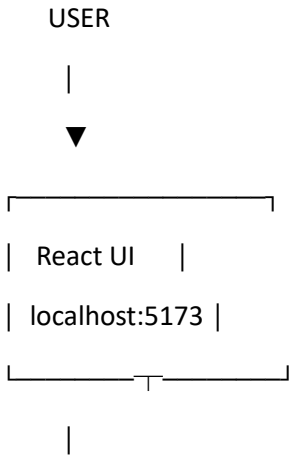
- confidence
- explanation
- retrieved evidence
- source URLs

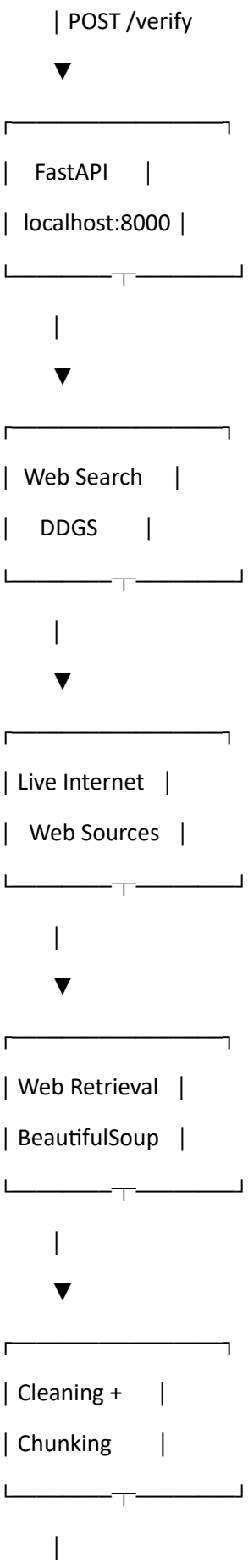
And now the verification result is also stored in **Amazon S3**.

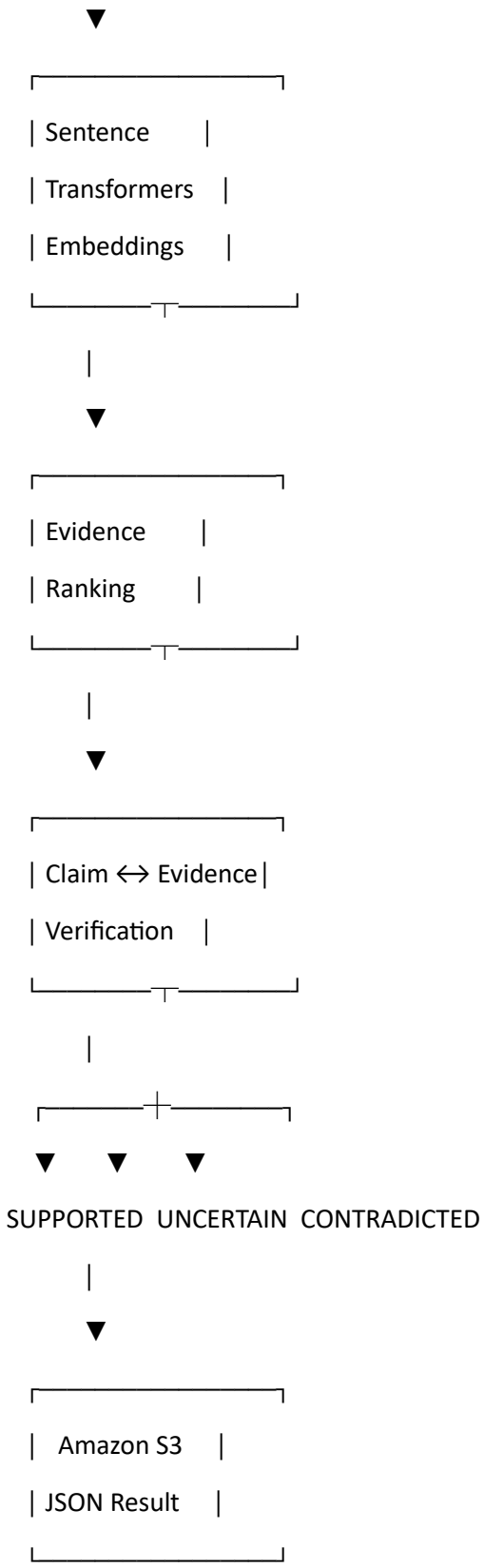
---

2. Complete VeriLingo pipeline

Your current system is basically:




| POST /verify
▼
FastAPI
localhost:8000
Web Search
DDGS
Live Internet
Web Sources
Web Retrieval
BeautifulSoup
Cleaning +
Chunking
|



Let's break that down.

---

3. Frontend — React

Your frontend is inside:

frontend/

You're using React + Vite.

The user sees your dark VeriLingo dashboard.

They enter:

India has won a FIFA World Cup

and click verify.

The frontend sends:

POST http://127.0.0.1:8000/verify

with:

```
{
  "claim": "India has won a FIFA World Cup"
}
```

The frontend then displays:

- verdict
- confidence
- explanation
- evidence
- similarity
- source quality
- source links

So React is basically the **presentation layer**.

---

4. FastAPI — backend/API layer

Your main.py is the controller for the application.

Currently:

React

↓

FastAPI

FastAPI receives the claim:

class ClaimRequest(BaseModel):

claim: str

Then:

evidence_results = retrieve_web_evidence(

data.claim,

max_sources=5,

top_k=5

)

So FastAPI tells your retrieval system:

"Find evidence for this claim."

Then:

result = verify_evidence(

data.claim,

evidence_results

)

Your verifier determines the result.

Finally:

save_verification(verification_result)

stores the complete result in AWS S3.

So main.py is essentially the **orchestrator**.

---

5. Live web search — DDGS

This is one of the most important parts of the project.

You're **not using a fixed database of facts**.

Instead:

User claim

↓

DDGS search

↓

Live web results

For example:

India has won a FIFA World Cup

can produce multiple live search results.

Your search system deliberately remains **generic**.

It doesn't contain logic like:

```
if "FIFA" in claim:
    search FIFA
```

because that would make the system domain-specific.

Instead, the **claim itself drives the search**.

---

6. Webpage retrieval

Search results give you:

title

URL

snippet

But a search snippet isn't always enough.

So VeriLingo attempts to retrieve the actual webpage using:

requests

+

BeautifulSoup

The webpage is cleaned by removing things such as:

script

style

nav

footer

header

ads/irrelevant UI

forms

iframes

Then the useful text is extracted.

This gives your system actual evidence instead of simply trusting the search result title.

---

7. Chunking

A webpage can contain thousands of words.

You don't want to compare the entire article with the claim as one giant piece.

So the text is divided into smaller chunks.

Conceptually:

Article

|

├— Chunk 1

├— Chunk 2

├— Chunk 3

├— Chunk 4

└— ...

Your current chunking uses approximately:

50 words per chunk

15-word overlap

The overlap helps prevent important information from being cut between two chunks.

---

8. Sentence Transformers

This is your main **semantic retrieval component**.

You use:

all-MiniLM-L6-v2

The model converts text into numerical vectors called **embeddings**.

For example:

Claim

"Earth has a moon"

↓

[0.12, -0.34, 0.81, ...]

and evidence:

"Earth's natural satellite is the Moon."

↓

[0.10, -0.30, 0.79, ...]

The system calculates **cosine similarity** between them.

High similarity means:

"This evidence is semantically relevant to the claim."

Important distinction:

Similarity ≠ truth

If the claim is:

"Earth has purple oceans."

and an article discusses:

"Purple Earth hypothesis..."

the text can be highly similar.

That doesn't mean the claim is true.

That's exactly why you added the next verification layer.

---

9. Evidence ranking

Each retrieved chunk gets:

similarity_score

source_quality

final_score

Your current final score is approximately:

75% semantic similarity

+

25% source-quality heuristic

So a highly relevant source from a recognized official/news domain can rank above an otherwise equally relevant low-quality source.

But importantly:

Source quality is a ranking heuristic, not proof that a claim is true.

---

10. Your verification engine

This is currently demo_verifier.py.

This is where the project becomes more than a simple semantic search engine.

It looks at:

A. Word overlap

How much of the claim's meaningful vocabulary appears in the evidence?

B. Semantic relevance

How similar is the evidence to the claim?

C. Negation

For example:

Claim:

India has won the FIFA World Cup.

Evidence:

India has never won the FIFA World Cup.

The semantic similarity is high.

But the **polarity is opposite**.

Therefore:

CONTRADICTED

D. Uncertainty

Your verifier also detects language such as:

could

might

possibly

hypothesis

theoretical

unproven

speculated

potential

This prevents something like:

"The purple oceans hypothesis proposes..."

from being treated as proof that:

"Earth has purple oceans."

So the result becomes:

UNCERTAIN

rather than incorrectly saying:

SUPPORTED

---

11. Your three important demonstrations

These are particularly useful because they demonstrate all three outcomes.

Test 1

Earth has a moon.

Evidence supports the claim.

Result:

SUPPORTED

Test 2

India has won a FIFA World Cup.

Retrieved evidence explicitly says India has never won it.

Result:

CONTRADICTED

Test 3

Earth has purple oceans.

The web contains related discussion/hypotheses, but no solid evidence establishing the claim.

Result:

UNCERTAIN

That's a much better demonstration than testing three obviously true claims.

---

12. Amazon S3 — your first AWS service

This is the AWS service you've already successfully integrated.

Your application now does:

Verification

↓

JSON

↓

Amazon S3

For example:

verifications/

20260920T144535Z.json

The JSON contains information such as:

{

"claim": "...",

"verdict": "CONTRADICTED",

"confidence": "HIGH",

"explanation": "...",

"retrieved_evidence": [...]

}

Why S3?

S3 is **object storage**.

It's appropriate for storing complete verification records and evidence snapshots.

You can explain:

"Amazon S3 provides durable cloud storage for VeriLingo's verification results and retrieved evidence."

And we've actually tested this—not just designed it.

You uploaded test data through the CLI and then successfully uploaded actual VeriLingo verification JSON through boto3.

---

13. What happens with AWS credentials?

Your Python code does **not** contain:

AWS_ACCESS_KEY

AWS_SECRET_KEY

Instead, you configured the AWS CLI locally:

aws configure

and boto3 automatically obtains the credentials from your AWS environment.

That's important from a security perspective.

And your GitHub .gitignore prevents things such as:

.aws/

.env

from being committed.

---

14. What AWS are we going to add?

This is where the project becomes a much more complete cloud architecture.

Stage 1 — DONE

Amazon S3

Purpose:

Persistent cloud storage

Status:

✅ **Implemented**

---

15. Stage 2 — AWS Lambda

This is our next major component.

Currently:

React

↓

localhost:8000

↓

FastAPI

After Lambda:

React

↓

API Gateway

↓

Lambda

↓

VeriLingo backend

Lambda provides **serverless compute**.

Instead of keeping a Python server running continuously, AWS runs your backend when requests arrive.

Conceptually:

User requests verification

↓

Lambda

↓

process claim

↓

return result

One complication

Your project uses:

Sentence Transformers

PyTorch/model dependencies

Those are relatively heavy.

Therefore we're planning to package VeriLingo as a **Lambda container image**, rather than trying to squeeze everything into a conventional Lambda ZIP package.

That's why Docker became the next prerequisite.

---

16. API Gateway

Once Lambda works, we'll put:

Amazon API Gateway

in front of it.

Currently:

POST localhost:8000/verify

Eventually:

POST https://....amazonaws.com/verify

Architecture:

React

|

▼

API Gateway

|

▼

Lambda

|

├── Web Search

├── Retrieval

├── Semantic Similarity

├── Verification

└── S3

This means your frontend no longer needs to talk to your laptop.

---

17. CloudWatch

Once Lambda is deployed, we'll use:

Amazon CloudWatch

for:

Lambda logs

Errors

Execution duration

Invocations

Monitoring

So if the verification function crashes, you can inspect the logs.

Architecture becomes:

Lambda

|

└──────► CloudWatch

|

├── Logs

├── Errors

└── Metrics

This gives your project an actual **monitoring/observability layer**.

---

18. DynamoDB

After Lambda/API Gateway, the next AWS component I'd consider is:

Amazon DynamoDB

S3 stores the **complete verification document**.

DynamoDB can store **query metadata**:

claim_id

claim

verdict

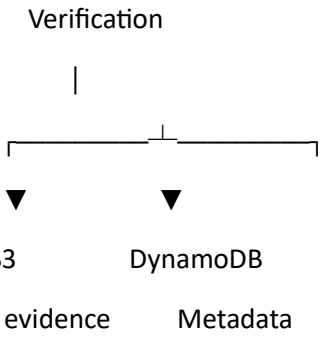
confidence

timestamp

source_count

s3_object_key

So:



This is a sensible cloud-storage architecture rather than storing everything in one place.

---

19. Amazon Translate

This one is particularly relevant because of **VeriLingo's multilingual purpose**.

Eventually:

Tamil/Hindi/etc.

↓

Amazon Translate

↓

Normalized English claim

↓

Web search

↓

Evidence

↓

Verification

For example:

Tamil claim

↓

Translate

↓

English representation

↓

Global web search

This would make the "Lingo" part of VeriLingo genuinely cloud-integrated.

---

20. Amazon Textract

Another optional feature is image/screenshot verification.

User uploads:

Screenshot of a news post

Then:

S3

↓

Textract

↓

Extract text

↓

VeriLingo verification

So VeriLingo wouldn't only accept typed claims.

It could eventually accept:

Text

Screenshot

PDF

and extract the claim before verification.

---

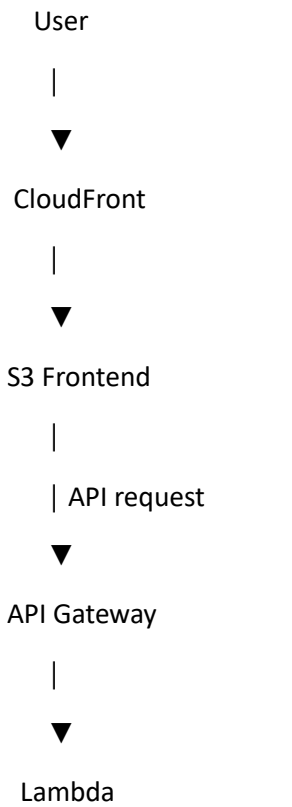
21. CloudFront + S3 for the frontend

Currently:

localhost:5173

Eventually we can deploy your React frontend.

Architecture:

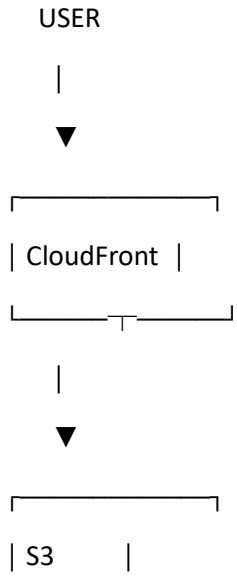


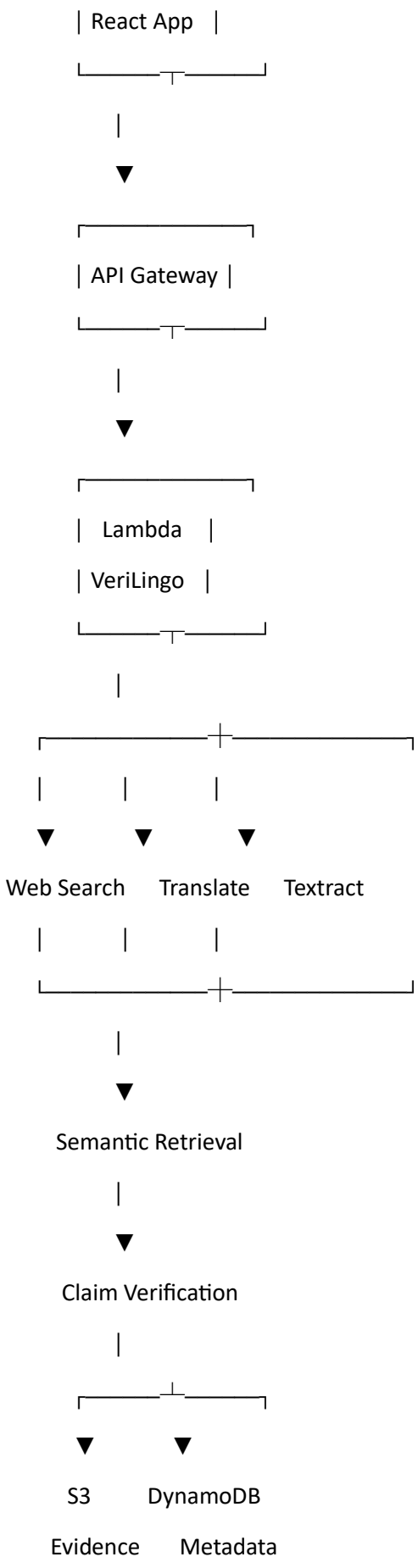
Then your entire application is accessible through the internet instead of requiring your laptop.

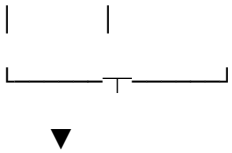
---

22. Final architecture we're aiming toward

The mature version would look roughly like:




React App
API Gateway
Lambda
VeriLingo
Web Search
Translate
Textract
Semantic Retrieval
Claim Verification
S3
Evidence
DynamoDB
Metadata



CloudWatch

Logs + Monitoring

---

23. AWS services: what we've done vs what we're planning

AWS technology	Role	Status
IAM	Identity & permissions	✅ Used
AWS CLI	Configure/test AWS access	✅ Used
Amazon S3	Store verification JSON/evidence	✅ Working
boto3	Python → AWS communication	✅ Working
AWS Lambda	Serverless backend	SOON Next
API Gateway	Public /verify API	SOON After Lambda
CloudWatch	Logs/monitoring	SOON With Lambda
DynamoDB	Verification metadata/history	SOON Optional/next
Amazon Translate	Multilingual claim processing	SOON Optional
Amazon Textract	Screenshot/PDF text extraction	SOON Optional
CloudFront	Frontend delivery/CDN	SOON Final deployment

What I would prioritize

Given that your core system is already working and you've backed it up on GitHub:

Now:

S3 ✅

↓

Lambda 🔥

↓

API Gateway 🔥

↓

CloudWatch 🔥

Then, if time allows:

DynamoDB

Translate

Textract

CloudFront

That gives you a strong story:

"VeriLingo is a cloud-based, multilingual, evidence-driven claim verification system. The frontend communicates with a serverless API through API Gateway. Lambda executes the verification pipeline, which performs live web retrieval and semantic evidence analysis. Verification results are persistently stored in S3, metadata can be indexed in DynamoDB, and CloudWatch provides observability."

Most importantly, **we won't add AWS services merely to increase the count**. Each service should have a clear architectural responsibility.

And your current local implementation remains the **reference version** while we gradually move pieces into AWS.